

[illegible]

[illegible]

CODE:

—

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

image = cv2.imread("/goat.jpg")

if image is None:
    print("Error: Could not load image. Please check the file path and ensure the image exists.")
else:
    #image_re_linear = cv2.resize(image, None, fx=1.5, fy=2.5, interpolation=cv2.INTER_LINEAR)
    #image_re_cubic = cv2.resize(image, None, fx=1.5, fy=2.5, interpolation=cv2.INTER_CUBIC)
    #matrix = np.float32([[1, 0, 20],[0, 1, 80]])
    #translated = cv2.warpAffine(image, matrix, (image.shape[1], image.shape[0]))
    height, weight = image.shape[:2]
    matrix1 = cv2.getRotationMatrix2D((0, 0), 45, 1)
    Rotation = cv2.warpAffine(image, matrix1, (weight, height))
    cv2_imshow(image)
    #cv2_imshow(image_re_linear)
    #cv2_imshow(image_re_cubic)
    #cv2_imshow(translated)
    cv2_imshow(Rotation)
```

OUTPUT-



Aim- Perform Image Stitching.

[illegible]

[illegible]

CODE:

—

```
from PIL import Image
from google.colab.patches import cv2_imshow
import numpy as np

image1 = Image.open("/content/A.jpg")
image2 = Image.open("/content/W.jpg")
image3 = Image.open("/content/Z.png")

image1 = image1.resize((500,600))
image2 = image2.resize((500,600))
image3 = image3.resize((500,600))

image1_size = image1.size
image2_size = image2.size
image3_size = image3.size

new_image = Image.new('RGB', (image1_size[0] + image2_size[0] +
image3_size[0],
image1_size[1]), (250,250,250))

#Paste the image to the new image
new_image.paste(image1, (0,0))
new_image.paste(image2, (image1_size[0],30))
new_image.paste(image3, (image1_size[0] + image2_size[0],50))

#show the final combined image
cv2_imshow(np.array(new_image))
```

OUTPUT-

BEFORE IMAGE STICHING:



AFTER IMAGE STICHING:



PRACTICAL NO. 3

***Aim-* Perform Camera Calibration.**

[illegible]

[illegible]

CODE:

```
# Import required modules
```

```

import cv2
import numpy as np
import os
import glob

# Define the dimensions of checkerboard
CHECKERBOARD = (6, 9)

# stop the iteration when specified
# accuracy, epsilon, is reached or
# specified number of iterations are completed.
criteria = (cv2.TERM_CRITERIA_EPS +
            cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)

# Vector for 3D points
threedpoints = []

# Vector for 2D points
twodpoints = []

# 3D points real world coordinates
objectp3d = np.zeros((1, CHECKERBOARD[0]
                     * CHECKERBOARD[1],
                     3), np.float32)
objectp3d[0, :, :2] = np.mgrid[0:CHECKERBOARD[0],
                                0:CHECKERBOARD[1]].T.reshape(-1, 2)
prev_img_shape = None

# Extracting path of individual image stored
# in a given directory. Since no path is
# specified, it will take current directory
# jpg files alone
images = glob.glob('*.jpg')

for filename in images:
    image = cv2.imread(filename)
    grayColor = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Find the chess board corners
    # If desired number of corners are
    # found in the image then ret = true
    ret, corners = cv2.findChessboardCorners(
        grayColor, CHECKERBOARD,
        cv2.CALIB_CB_ADAPTIVE_THRESH
        + cv2.CALIB_CB_FAST_CHECK +
        cv2.CALIB_CB_NORMALIZE_IMAGE)

    # If desired number of corners can be detected then,
    # refine the pixel coordinates and display
    # them on the images of checker board

```

```

if ret == True:
    threedpoints.append(objectp3d)

    # Refining pixel coordinates
    # for given 2d points.
    corners2 = cv2.cornerSubPix(
        grayColor, corners, (11, 11), (-1, -1), criteria)

    twodpoints.append(corners2)

    # Draw and display the corners
    image = cv2.drawChessboardCorners(image,
        CHECKERBOARD,
        corners2, ret)

    cv2.imshow('img', image)
    cv2.waitKey(0)

cv2.destroyAllWindows()

h, w = image.shape[:2]

# Perform camera calibration by
# passing the value of above found out 3D points (threedpoints)
# and its corresponding pixel coordinates of the
# detected corners (twodpoints)
ret, matrix, distortion, r_vecs, t_vecs = cv2.calibrateCamera(
    threedpoints, twodpoints, grayColor.shape[:-1], None, None)

# Displaying required output
print(" Camera matrix:")
print(matrix)

print("\n Distortion coefficient:")
print(distortion)

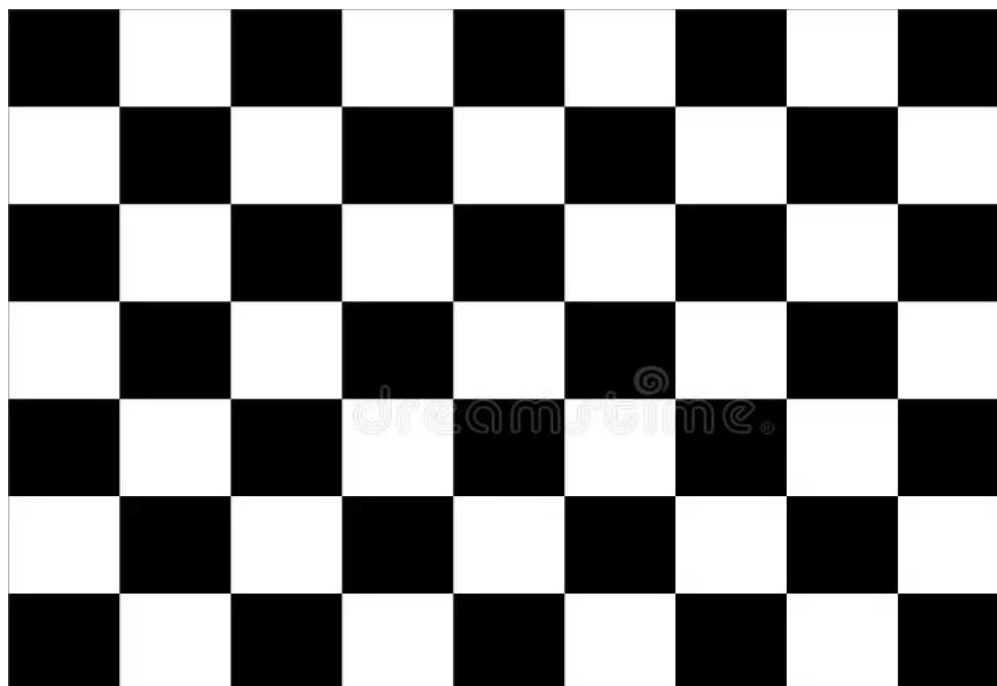
print("\n Rotation Vectors:")
print(r_vecs)

print("\n Translation Vectors:")
print(t_vecs)

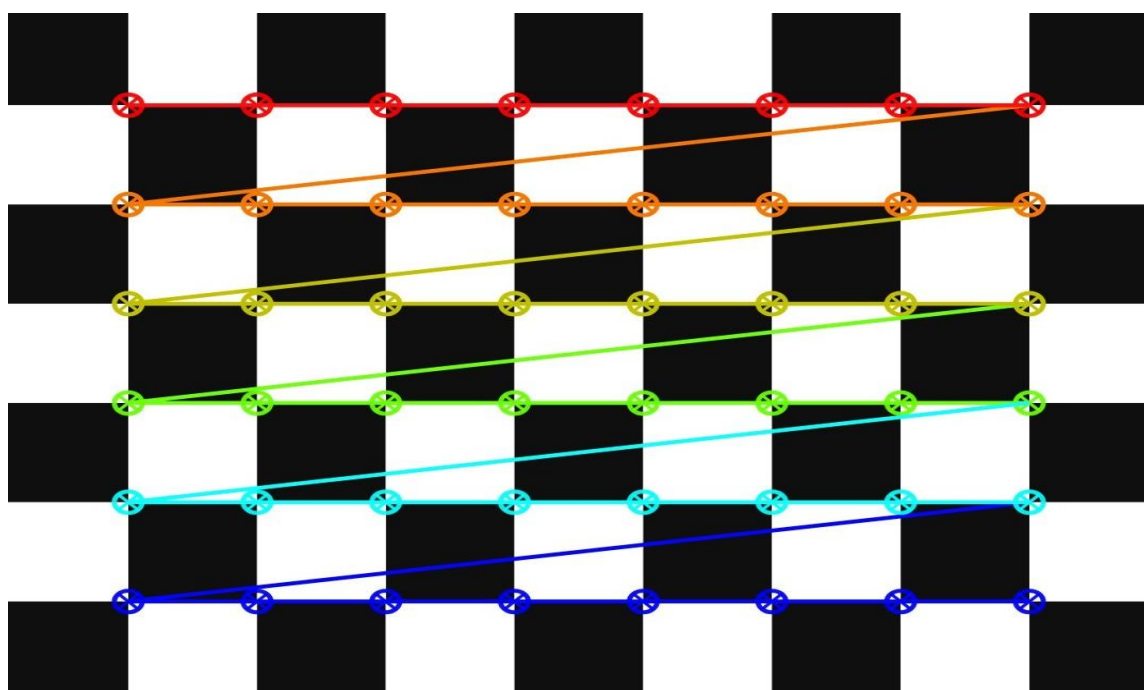
```

OUTPUT:

ORIGINAL IMAGE



AFTER CORNER DETECTION



PRACTICAL NO. 4

Aim- Perform the following:

- Face detection
- Object detection
- Pedestrian detection
- Face Recognition

WRITEUP

[illegible]

a. Face detection

CODE:

```
import cv2
from google.colab.patches import cv2_imshow

# Load Haar Cascade
face_detector = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
)

# Read image
image = cv2.imread("/content/photo.jpg")

if image is None:
    print("Image not found!")
else:
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

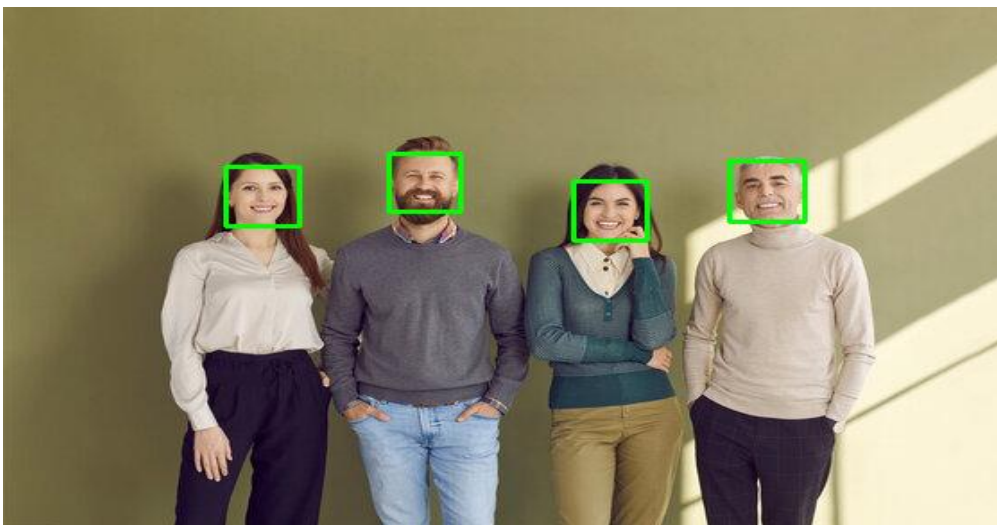
    faces = face_detector.detectMultiScale(
        gray,
        scaleFactor=1.1,
        minNeighbors=5,
        minSize=(30, 30)
    )

    print("Faces detected:", len(faces))

    for (x, y, w, h) in faces:
        cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 2)

    cv2_imshow(image)
```

OUTPUT:



[illegible]

[illegible]

CODE:

```
import cv2
from matplotlib import pyplot as plt
# Paths
image_path = "/content/stop.webp"
cascade_path = "/content/stop_sign_classifier_2.xml"
# Load image
img = cv2.imread(image_path)
if img is None:
    raise FileNotFoundError("Image file could not be loaded. Check the
path.")
# Convert image
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
# Load cascade
stop_data = cv2.CascadeClassifier(cascade_path)
if stop_data.empty():
    raise FileNotFoundError("Cascade file could not be loaded. Check the
path.")
# Detect and draw rectangles
for (x, y, w, h) in stop_data.detectMultiScale(img_gray, minSize=(20,
20)):
    cv2.rectangle(img_rgb, (x, y), (x + h, y + w), (0, 255, 0), 3)
# Display image
plt.imshow(img_rgb)
plt.show()
```

OUTPUT:



[illegible]

CODE

```
import cv2
import imutils
from google.colab.patches import cv2_imshow

# Initializing the HOG person detector
hog = cv2.HOGDescriptor()
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())

# Video capture from file
cap = cv2.VideoCapture("/content/video.mp4")

# Check if video opened successfully
if not cap.isOpened():
    print("Error: Could not open video file.")
else:
    while True:
        ret, image = cap.read()
        if not ret:
            break
        image = imutils.resize(image, width=min(400, image.shape[1]))
        regions, _ = hog.detectMultiScale(image, winStride=(4, 4),
padding=(4, 4), scale=1.05)
        for (x, y, w, h) in regions:
            cv2.rectangle(image, (x, y), (x + w, y + h), (0, 0, 255), 2)
        cv2_imshow(image)
    # cv2_imshow does not require waitKey or destroyAllWindows in Colab
    cap.release()
```

OUTPUT:



[illegible]

CODE:

```
import cv2

import imutils

from google.colab.patches import cv2_imshow

hog = cv2.HOGDescriptor()
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())

cap = cv2.VideoCapture("/content/face video.mp4")

while cap.isOpened():
    ret, image = cap.read()

    if not ret:
        break

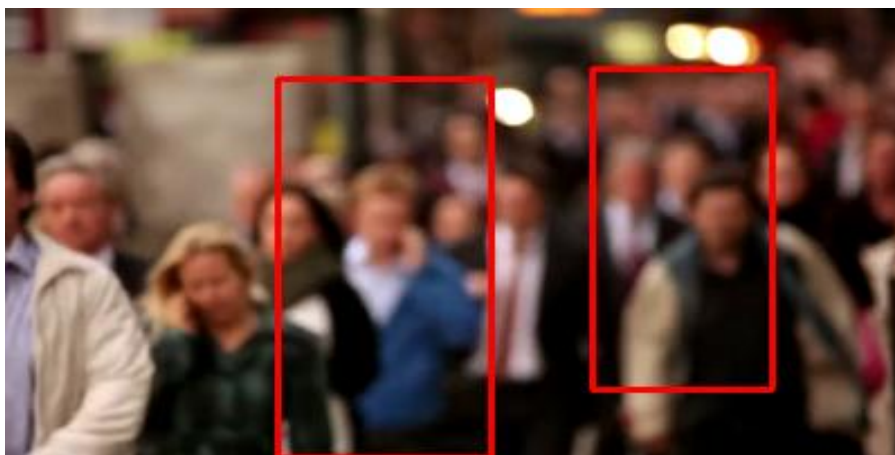
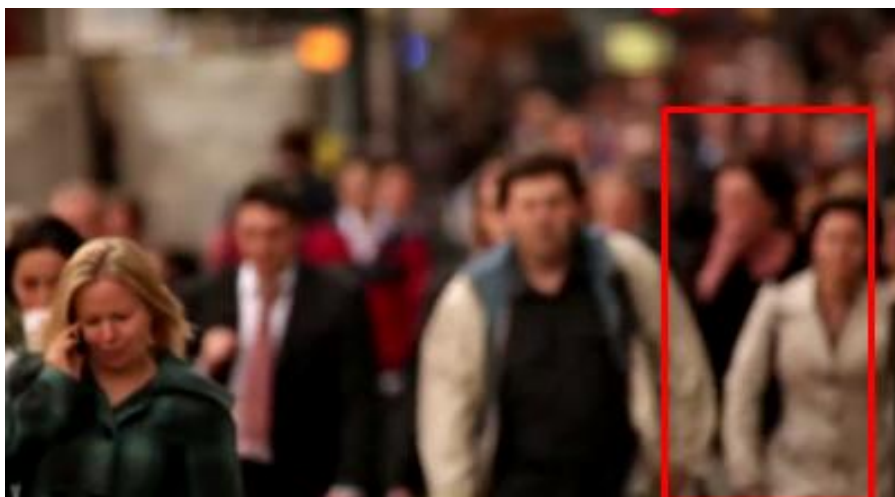
    image = imutils.resize(image, width=min(400, image.shape[1]))
    regions, _ = hog.detectMultiScale(image, winStride=(4, 4),
padding=(4, 4), scale=1.05)

    for (x, y, w, h) in regions:
        cv2.rectangle(image, (x, y), (x + w, y + h), (0, 0, 255), 2)

    cv2_imshow(image)

# Release video capture
cap.release()
```

OUTPUT:



PRACTICAL NO. 5

Aim- Construct 3D model from images.

WRITEUP

[illegible]

[illegible]

CODE:

```
from PIL import Image
import numpy as np
from IPython.display import display

def shift_image(img, depth_img, shift_amount=10):

    # Convert images
    img = img.convert("RGBA")
    depth_img = depth_img.convert("L")

    # Make sure both images have the same size
    depth_img = depth_img.resize(img.size)

    # Convert to NumPy arrays
    data = np.array(img)
    depth_data = np.array(depth_img)

    # Calculate horizontal shift for each pixel
    deltas = ((depth_data / 255.0) * shift_amount).astype(np.int32)

    # Create empty output image
    shifted_data = np.zeros_like(data)

    height, width, channels = data.shape

    # Shift pixels
    for y in range(height):
        for x in range(width):
            dx = deltas[y, x]
            new_x = x + dx

            if 0 <= new_x < width:
                shifted_data[y, new_x] = data[y, x]

    # Convert back to PIL Image
    shifted_image = Image.fromarray(shifted_data)

    return shifted_image

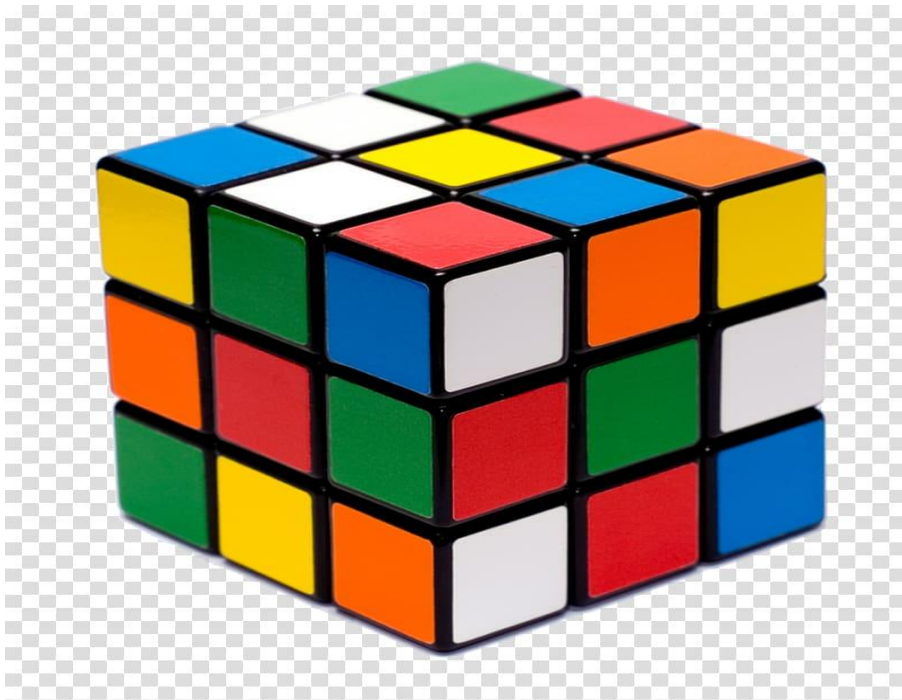
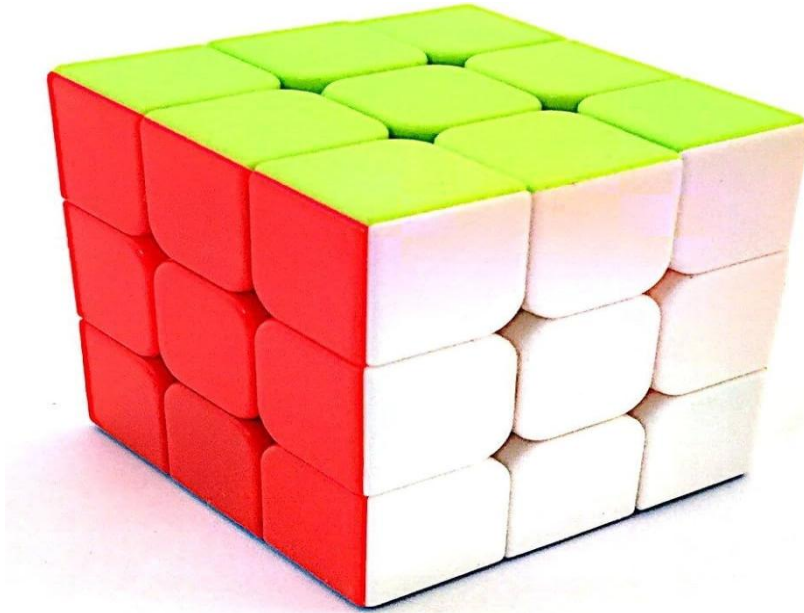
# Load images
img1 = Image.open("/content/cube image 01.jpg")
img2 = Image.open("/content/cube image 02.jpg")

# ---- First cube ----
# Shift cube 1 using cube 2 as its depth map
shifted_img1 = shift_image(img1, img2, shift_amount=10)

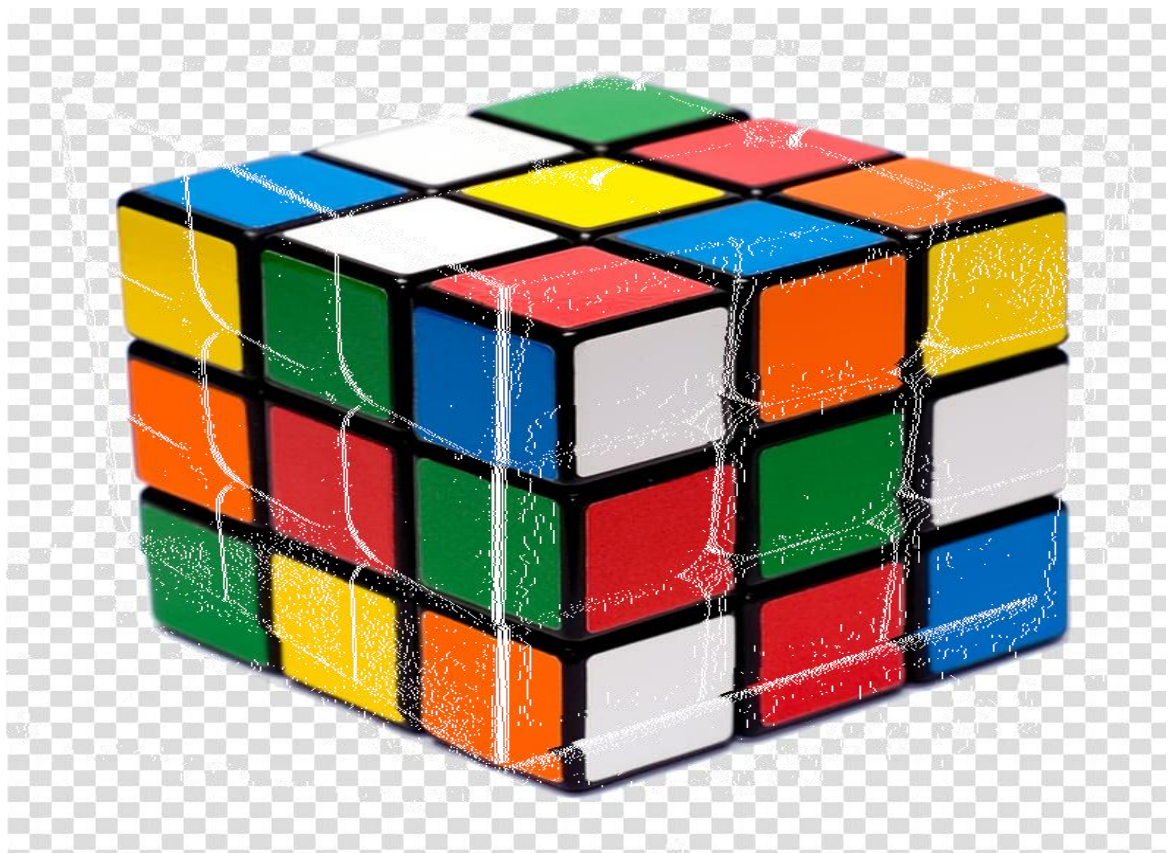
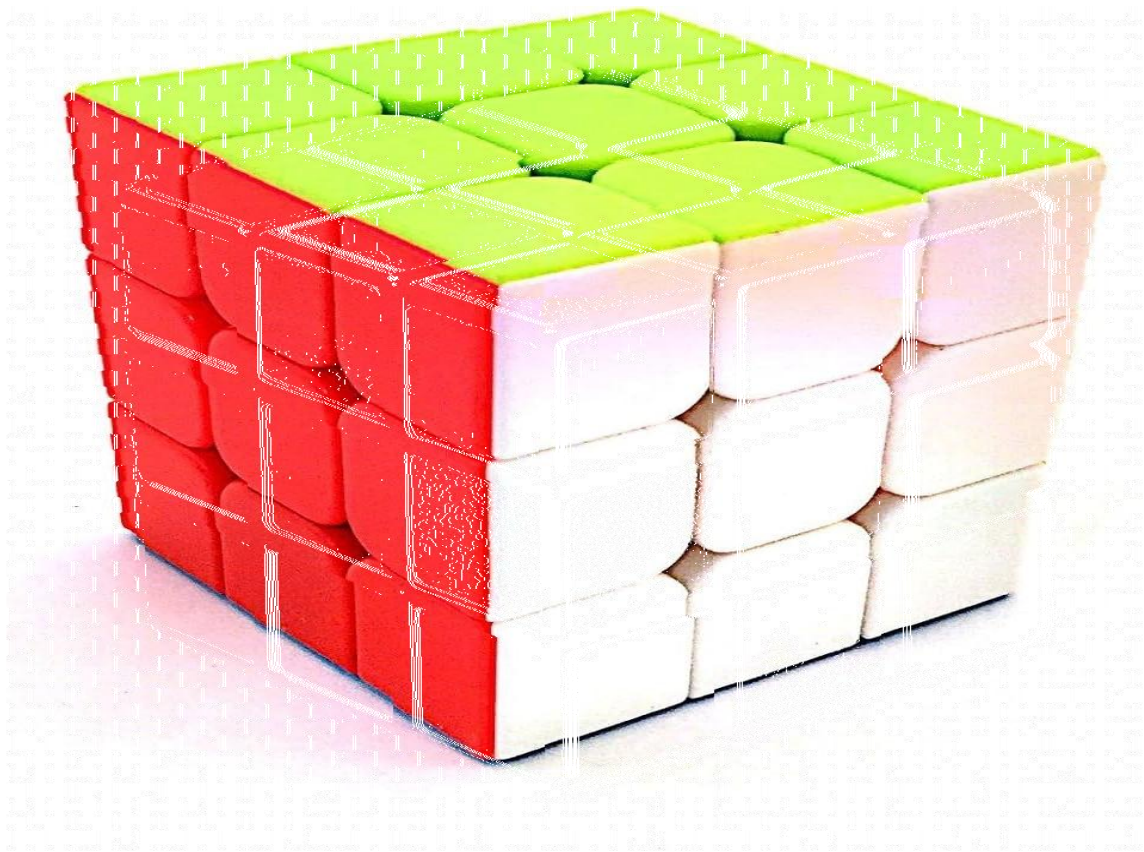
# Save output
shifted_img1.save("/content/output1.png")
# Display output
display(shifted_img1)
-
```

```
# ---- Second cube ----  
# Shift cube 2 using cube 1 as its depth map  
shifted_img2 = shift_image(img2, img1, shift_amount=10)  
# Save output  
shifted_img2.save("/content/output2.png")  
# Display output  
display(shifted_img2)
```

INPUT Images:



OUTPUT Model:



PRACTICAL NO. 6

Aim- Implement object detection and tracking from video.

WRITEUP

[illegible]

[illegible]

CODE:

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
-
```

```

# Load YOLO model
net = cv2.dnn.readNet("yolov3.weights", "yolov3.cfg")

# Load class names
classes = []
with open("coco.names", "r") as f:
    classes = [line.strip() for line in f.readlines()]

# Get output layers
layer_names = net.getLayerNames()
output_layers = [layer_names[i - 1] for i in net.getUnconnectedOutLayers()]

# Bounding box color
colors = [(0, 255, 0)]

# Load video
cap = cv2.VideoCapture("/content/Mumbai video.mp4")

# Check video
if not cap.isOpened():
    print("Error: Cannot open video file")
    exit()

# Tracking variables
tracked_objects = {}
next_id = 1

frame_count = 0

while True:
    ret, frame = cap.read()

    if not ret:
        print("Video processing completed.")
        break

    frame_count += 1

    height, width, channels = frame.shape

    # Create blob
    blob = cv2.dnn.blobFromImage(
        frame,
        0.00392,
        (416, 416),
        (0, 0, 0),
        True,
        crop=False
    )

    net.setInput(blob)
    outs = net.forward(output_layers)

    boxes = []
    confidences = []
    class_ids = []

    # Detect objects
    for out in outs:
        for detection in out:

            scores = detection[5:]
            class_id = np.argmax(scores)
            confidence = scores[class_id]

            if confidence > 0.5:

```



```

        center_x = int(detection[0] * width)
        center_y = int(detection[1] * height)

        w = int(detection[2] * width)
        h = int(detection[3] * height)

        x = int(center_x - w / 2)
        y = int(center_y - h / 2)

        boxes.append([x, y, w, h])
        confidences.append(float(confidence))
        class_ids.append(class_id)

# Non-Maximum Suppression
indexes = cv2.dnn.NMSBoxes(
    boxes,
    confidences,
    0.5,
    0.4
)

# Draw boxes and track objects
if len(indexes) > 0:

    for idx in indexes:

        if isinstance(idx, (list, tuple, np.ndarray)):
            i = idx[0]
        else:
            i = idx

        x, y, w, h = boxes[i]

        label = classes[class_ids[i]]
        color = colors[0]

        matched_id = None

        for obj_id, tracked_box in tracked_objects.items():

            if abs(x - tracked_box[0]) < 50 and abs(y - tracked_box[1]) < 50:
                matched_id = obj_id
                break

        if matched_id is not None:

            tracked_objects[matched_id] = (x, y)

            cv2.rectangle(
                frame,
                (x, y),
                (x + w, y + h),
                color,
                2
            )

            cv2.putText(
                frame,
                f"{label} {matched_id}",
                (x, y - 10),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.5,
                color,
                2
            )

        else:

```

```
        tracked_objects[next_id] = (x, y)

        cv2.rectangle(
            frame,
            (x, y),
            (x + w, y + h),
            color,
            2
        )

        cv2.putText(
            frame,
            f"{label} {next_id}",
            (x, y - 10),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.5,
            color,
            2
        )

        next_id += 1

# Display every 30th frame in Colab
if frame_count % 30 == 0:
    cv2.imshow(frame)

cap.release()

print("Processing Finished.")
```

OUTPUT:



This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

CODE:

```
from google.colab.patches import cv2_imshow
import cv2
import numpy as np

def extract_features_and_match(img1, img2):
    # Create SIFT detector
    sift = cv2.SIFT_create()

    # Detect keypoints and descriptors
    kp1, des1 = sift.detectAndCompute(img1, None)
    kp2, des2 = sift.detectAndCompute(img2, None)

    # BF Matcher
    bf = cv2.BFMatcher()
    matches = bf.knnMatch(des1, des2, k=2)

    # Lowe's Ratio Test
    good_matches = []
    for m, n in matches:
        if m.distance < 0.75 * n.distance:
            good_matches.append(m)

    return kp1, kp2, good_matches

def apply_ransac(kp1, kp2, matches):
    if len(matches) < 4:
        print("Not enough matches found!")
        return None, None

    src_pts = np.float32(
        [kp1[m.queryIdx].pt for m in matches]
    ).reshape(-1, 1, 2)

    dst_pts = np.float32(
        [kp2[m.trainIdx].pt for m in matches]
    ).reshape(-1, 1, 2)

    M, mask = cv2.findHomography(
        src_pts,
        dst_pts,
        cv2.RANSAC,
        5.0
    )

    return M, mask

def draw_matches_with_mask(img1, kp1, img2, kp2, matches, mask):
    if mask is None:
        return cv2.drawMatches(
            img1, kp1,
```

```

        img2, kp2,
        matches, None,
        flags=2
    )

    matches_mask = mask.ravel().tolist()

    draw_params = dict(
        matchColor=(0, 255, 0),
        singlePointColor=None,
        matchesMask=matches_mask,
        flags=2
    )

    img_matches = cv2.drawMatches(
        img1, kp1,
        img2, kp2,
        matches, None,
        **draw_params
    )

    return img_matches

# Load images
img1 = cv2.imread("/content/packet chips.jpg", cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread("/content/packet.png", cv2.IMREAD_GRAYSCALE)

# Check images loaded successfully
if img1 is None:
    print("Error: packet.jpg not found")
elif img2 is None:
    print("Error: pac.jpg not found")
else:
    # Feature matching
    kp1, kp2, matches = extract_features_and_match(img1, img2)

    print("Good Matches Found:", len(matches))

# RANSAC
M, mask = apply_ransac(kp1, kp2, matches)

# Draw results
img_matches = draw_matches_with_mask(
    img1, kp1,
    img2, kp2,
    matches, mask
)

# Display in Colab
cv2.imshow(img_matches)

```

OUTPUT:



This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

[illegible]

CODE:

```
# Image Colorization using OpenCV DNN (Google Colab)

from google.colab.patches import cv2_imshow
import cv2
import numpy as np
import os

# -----
# File Paths
# -----
proto_file = "/content/colorization_deploy_v2.prototxt"
model_file = "/content/colorization_release_v2.caffemodel"
hull_pts = "/content/pts_in_hull.npy"
img_path = "/content/image.jpg"

# -----
# Check Files
# -----
files = [proto_file, model_file, hull_pts, img_path]

for file in files:
    if not os.path.exists(file):
        print(f'Error: File not found -> {file}')
        raise FileNotFoundError(file)

print("All files found successfully!")

# -----
# Load Image
# -----
img = cv2.imread(img_path)

if img is None:
    raise ValueError("Unable to load image!")

# -----
# Preprocessing
# -----
scaled = img.astype("float32") / 255.0
lab_img = cv2.cvtColor(scaled, cv2.COLOR_BGR2LAB)

# -----
# Load Pre-trained Model
# -----
net = cv2.dnn.readNetFromCaffe(proto_file, model_file)

# Load cluster centers
pts = np.load(hull_pts)

# Populate model layers
class8 = net.getLayerId("class8_ab")
-
```

```

conv8 = net.getLayerId("conv8_313_rh")

pts = pts.transpose().reshape(2, 313, 1, 1)

net.getLayer(class8).blobs = [pts.astype("float32")]
net.getLayer(conv8).blobs = [np.full((1, 313), 2.606, dtype="float32")]

# -----
# Prepare Input
# -----
resized = cv2.resize(lab_img, (224, 224))

L = cv2.split(resized)[0]
L -= 50

net.setInput(cv2.dnn.blobFromImage(L))

# -----
# Predict ab Channels
# -----
ab = net.forward()[0, :, :, :].transpose((1, 2, 0))

ab = cv2.resize(
    ab,
    (img.shape[1], img.shape[0])
)

# -----
# Reconstruct Color Image
# -----
L_original = cv2.split(lab_img)[0]

colorized = np.concatenate(
    (L_original[:, :, np.newaxis], ab),
    axis=2
)

colorized = cv2.cvtColor(
    colorized,
    cv2.COLOR_LAB2BGR
)
colorized = np.clip(colorized, 0, 1)
colorized = (255 * colorized).astype("uint8")
# -----
# Display Results
# -----
original_display = cv2.resize(img, (400, 400))
colorized_display = cv2.resize(colorized, (400, 400))
result = np.hstack((original_display, colorized_display))
print("Left: Original Image")
print("Right: Colorized Image")

cv2.imshow(result)

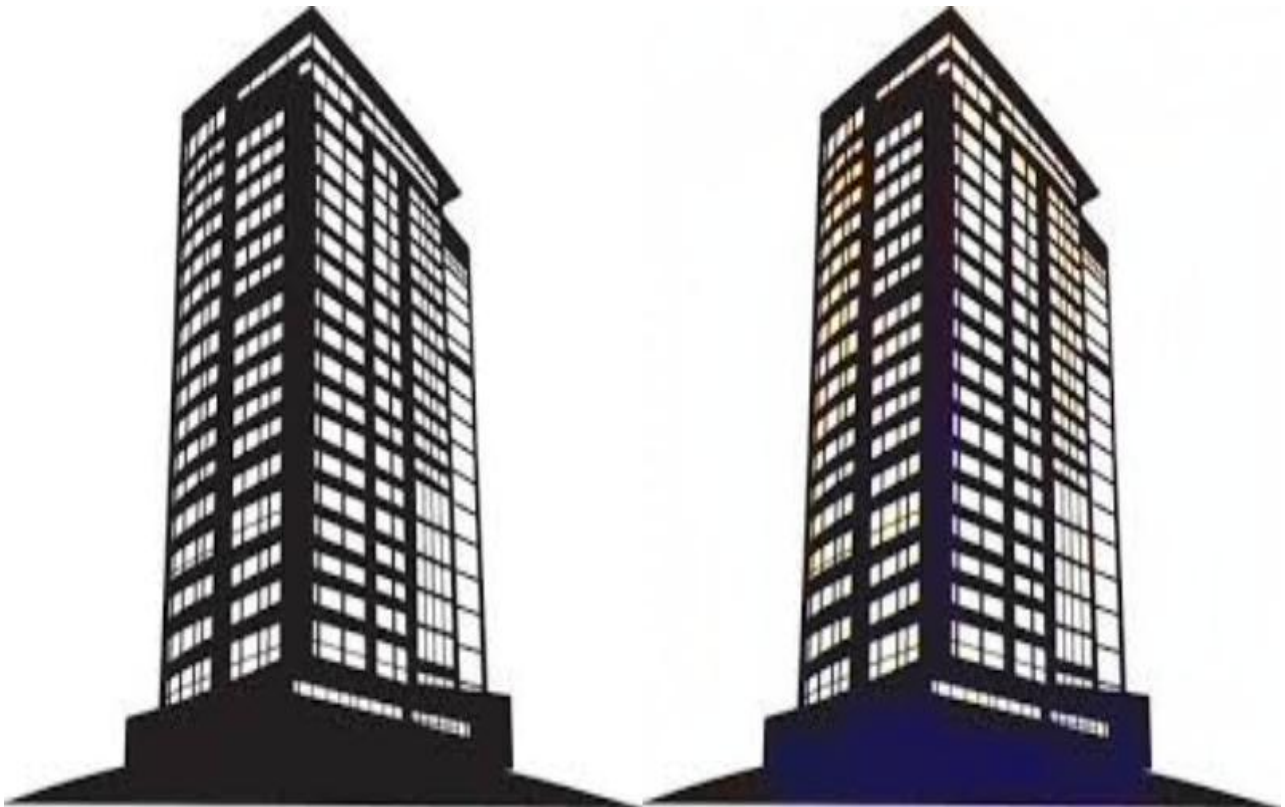
```

OUTPUT:

All files found successfully!

Left: Original Image

Right: Colorized Image



WRITEUP

[illegible]

[illegible]

CODE:

```
import cv2
import pytesseract

# Image path
image_path = "/content/image paragraph.png"

# Read image
img = cv2.imread(image_path)

if img is None:
    print("Error: Image not found!")
else:
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    _, thresh = cv2.threshold(
        gray, 0, 255,
        cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
    )

    rect_kernel = cv2.getStructuringElement(
        cv2.MORPH_RECT, (18, 18)
    )

    dilation = cv2.dilate(
        thresh,
        rect_kernel,
        iterations=1
    )

    contours, _ = cv2.findContours(
        dilation,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE
    )

    with open("recognized.txt", "w") as file:
        for cnt in contours:
            x, y, w, h = cv2.boundingRect(cnt)

            cropped = img[y:y+h, x:x+w]

            text = pytesseract.image_to_string(cropped)

            print("Recognized Text:")
            print(text)

            file.write(text + "\n")

    print("Text saved to recognized.txt")
```

OUTPUT:

Recognized Text:

As you know, we use electronics to process freight and documentation. We are in the process of having terminals placed in the export departments of some of our major customers around the country so they may keep track of all their shipments within our system. I would like to propose a similar tracking mechanism for your company. We could handle all of your export traffic from your locations around the country and monitor these exports with a terminal located in your home office. This could have many advantages for you. You could generate an export invoice in your export department, which could be transmitted via the computer to our office. You could trace your shipments more readily. This would all you to determine rating fees more accurately. Finally, your accounting department would benefit. All in all, your export operations would achieve greater efficiency.

Recognized Text:

You can achieve clarity and conciseness if you limit the length of your paragraphs. An excessively long paragraph (beyond six typed lines) requires too much work for your reader. Revise the following paragraph to make it more reader-friendly. Type the revised paragraph below the paragraph to be corrected.

Recognized Text:

Limiting Paragraph Length

Text saved to recognized.txt

A. Perform Image matting and compositing

[illegible]

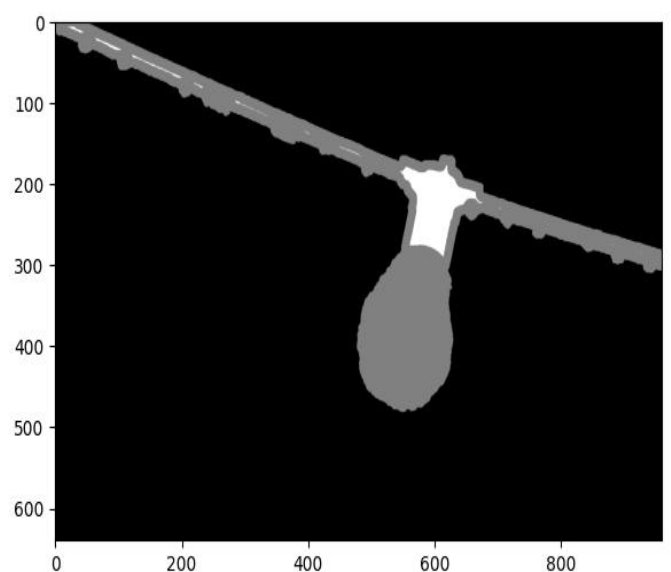
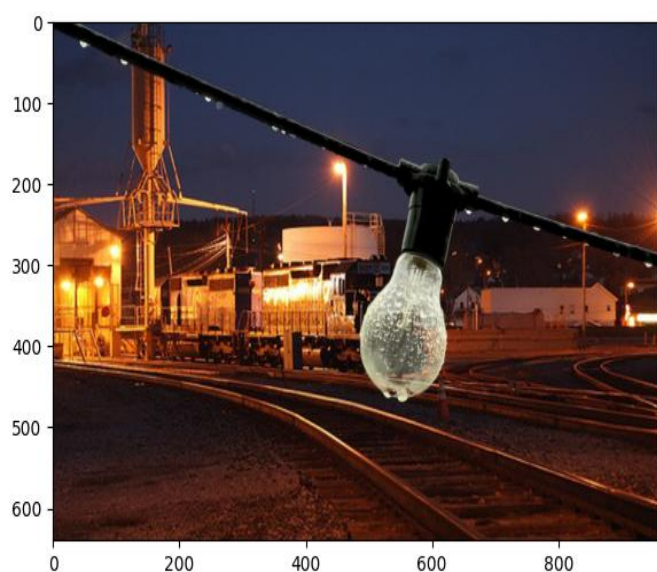
CODE:

```
import matplotlib.pyplot as plt
from PIL import Image
import requests

# Load the image and trimap
url = "https://github.com/hustvl/ViTMatte/blob/main/demo/bulb_rgb.png?raw=true"
image = Image.open(requests.get(url, stream=True).raw).convert("RGB")
url = "https://github.com/hustvl/ViTMatte/blob/main/demo/bulb_trimap.png?raw=true"
trimap = Image.open(requests.get(url, stream=True).raw)

# Display the image and trimap
plt.figure(figsize=(15, 15))
plt.subplot(1, 2, 1)
plt.imshow(image)
plt.subplot(1, 2, 2)
plt.imshow(trimap)
plt.show()
```

OUTPUT:



[illegible]

—

```
from PIL import Image

# Load images and convert to mode 'L'

im1 = Image.open("/content/elephant image.jpg").convert('L')
im2 = Image.open("/content/horse_1920x1080.png").convert('L')
mask = Image.open("/content/white_circle_mask.png").convert('L')


# Ensure all images are the same size

size = im1.size

im2 = im2.resize(size)
mask = mask.resize(size)


# Composite the images

im3 = Image.composite(im1, im2, mask)


# Show the result

from google.colab.patches import cv2_imshow

import numpy as np

import cv2

img = np.array(im3)

cv2_imshow(img)
```

OUTPUT:

